Buenafe, Ashley Dyriel V.
Lobo, Rendel Gion B.
Sison, Anoucshka Ysabeli A.

Week 2 Checkpoint Report
Energy Demand Forecasting + RL Load Scheduling

Overview
This checkpoint presents the initial implementation of an intelligent energy demand forecasting and load scheduling system. The project integrates time-series forecasting and reinforcement learning to predict electricity consumption and optimize load scheduling decisions. At this stage, the data pipeline, baseline models, and initial system components (CNN, NLP, and RL) have been implemented and tested.

Data and Processing
The project utilizes the UCI Individual Household Electric Power Consumption Dataset. Raw minute-level data was preprocessed and aggregated into hourly intervals to reduce noise and improve modeling efficiency.

Data preprocessing steps include:
- Combining date and time into a unified datetime format
- Converting all relevant features to numeric values
- Handling missing values using forward filling
- Resampling data into hourly averages

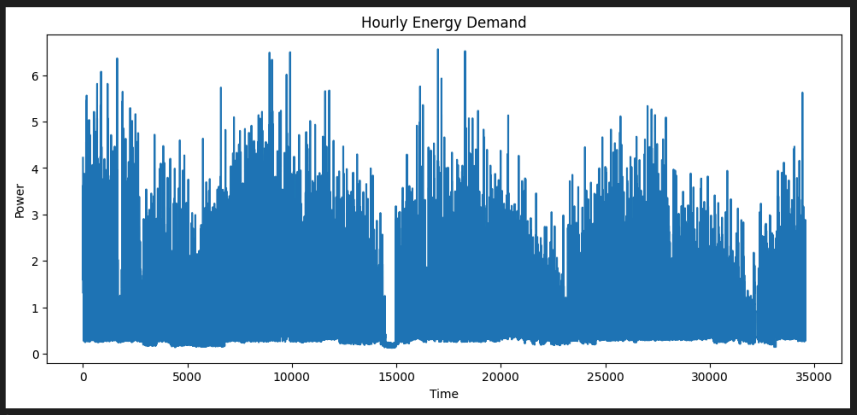
The final processed dataset contains 34,589 hourly samples with 7 numerical features.

The dataset was split chronologically into:
- Training set (70%)
- Validation set (15%)
- Test set (15%)

Exploratory Data Analysis
An exploratory data analysis (EDA) notebook was created to visualize energy demand trends and distributions.

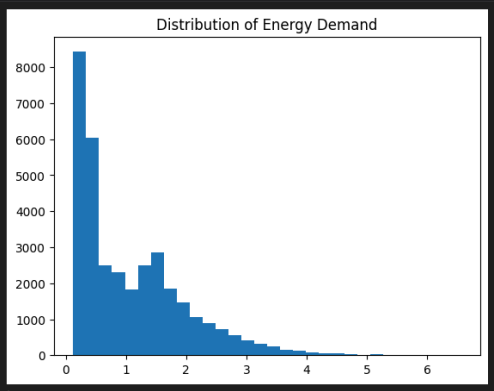
```python
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))
plt.plot(df["Global_active_power"])
plt.title("Hourly Energy Demand")
plt.xlabel("Time")
plt.ylabel("Power")
plt.show()
```

[5] ✓ 2.3s Python




```python
plt.hist(df["Global_active_power"], bins=30)
plt.title("Distribution of Energy Demand")
plt.show()
```

[6] ✓ 0.0s Python



Baseline Models and Initial Metrics

A simple Linear Regression model was implemented using a lag-1 feature to predict future energy demand.

Baseline Result:

- Mean Absolute Error (MAE): 0.3925

A deep learning model using a 1D CNN (TCN-style architecture) was also implemented to capture temporal patterns in the time series.

CNN/TCN Result:

- MAE: 0.5524
- MAPE: 0.9576

The deep learning model currently underperforms the baseline, indicating the need for further tuning and feature improvements.

The current results highlight that simpler models outperform deep learning in early stages, suggesting that feature engineering and hyperparameter tuning are critical next steps before scaling model complexity.

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# create lag feature
df["lag1"] = df["Global_active_power"].shift(1)

# remove missing values
df = df.dropna()

# features and target
X = df[["lag1"]]
y = df["Global_active_power"]

# split (80% train, 20% test)
split = int(len(df) * 0.8)

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# train model
model = LinearRegression()
model.fit(X_train, y_train)

# predict
y_pred = model.predict(X_test)

# evaluate
mae = mean_absolute_error(y_test, y_pred)

print("MAE:", mae)
```

[8] ✓ 5.6s Python

```
MAE: 0.39253607223715575
```

System Components

CNN Component (Time-Series Forecasting)

A convolutional neural network (TCN-style model) was implemented using 1D convolution layers. The model takes a sequence of past 24 hourly demand values and predicts the next time step. Initial training was completed with early results obtained.

src > models > tcn_model.py > SimpleTCN > __init__

```python
import torch
import torch.nn as nn

class SimpleTCN(nn.Module):
    def __init__(self, input_channels=1, hidden_channels=16, kernel_size=3):
        super(SimpleTCN, self).__init__()
        self.conv1 = nn.Conv1d(input_channels, hidden_channels, kernel_size)
        self.relu = nn.ReLU()
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(hidden_channels, 1)

    def forward(self, x):
        # x shape: (batch, channels, seq_len)
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x).squeeze(-1)
        x = self.fc(x)
        return x
```

NLP Component

An auxiliary NLP module was prototyped using TF-IDF vectorization and Logistic Regression. The component classifies textual descriptions of energy demand trends (e.g., increase, decrease, stable). This module is intended to support future integration with metadata or textual inputs.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

texts = [
    "Energy demand increased",
    "Power usage decreased",
    "Electricity demand is high",
    "Energy consumption dropped",
    "Electricity usage is stable"
]

labels = ["increase", "decrease", "increase", "decrease", "neutral"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)

model = LogisticRegression()
model.fit(X, labels)

# test prediction
sample = ["power demand increased significantly"]
prediction = model.predict(vectorizer.transform(sample))

print("Prediction:", prediction[0])
print("NLP prototype ready")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

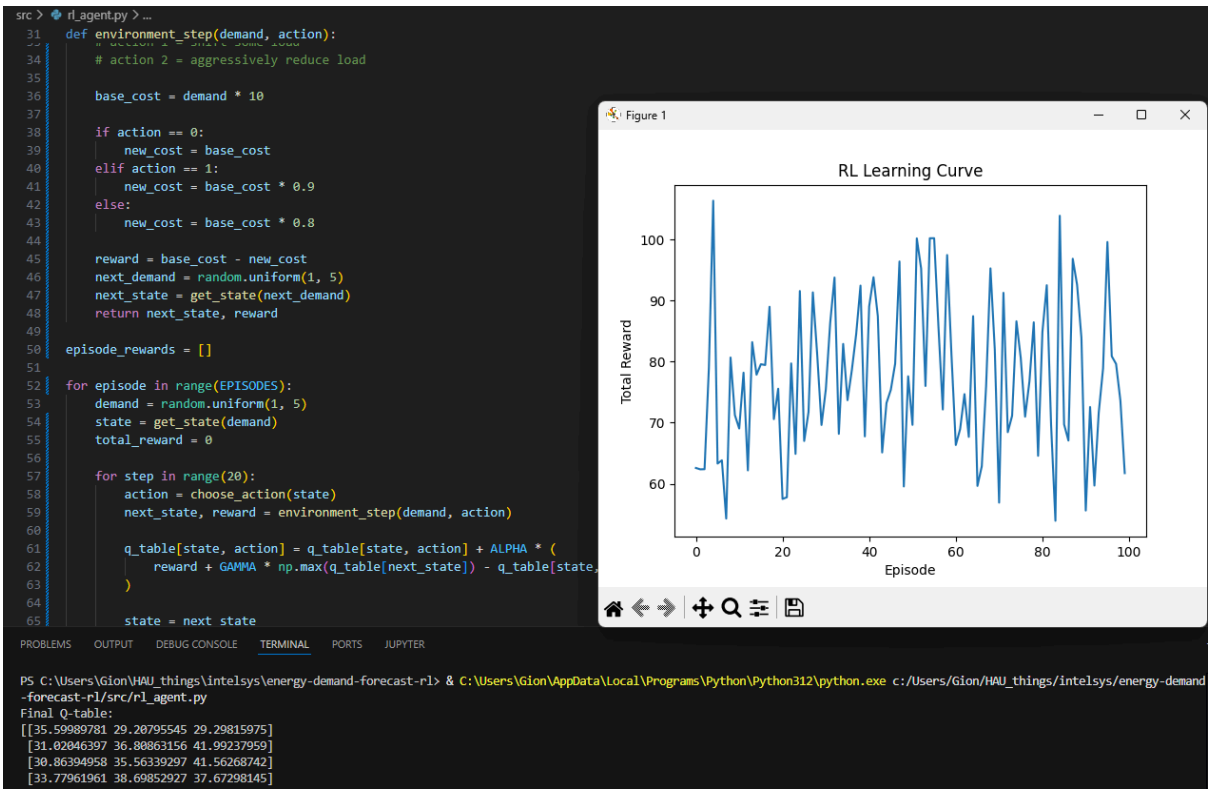
```
PS C:\Users\Gion\HAU_things\intelsys\energy-demand-forecast-rl> & C:\Users\Gion\AppData\Local\
-forecast-rl/src/nlp_classifier.py
Prediction: increase
NLP prototype ready
PS C:\Users\Gion\HAU_things\intelsys\energy-demand-forecast-rl>
```

Reinforcement Learning Component

A basic Q-learning agent was implemented as a scheduling prototype. The agent operates in a simplified environment where:

- State represents demand levels
- Actions represent load management strategies
- Reward is defined as a proxy for cost reduction

Initial learning behavior was observed through Q-table updates and reward accumulation across episodes.



Ethics and Model Card (Draft Status)

Initial drafts of the Model Card and Ethics Statement have been started. Key considerations include:

- Fair distribution of energy usage across users
- User autonomy in overriding automated decisions
- Transparency of system recommendations
- Avoiding over-reliance on automated scheduling

These ethical considerations are being incorporated into the system design and will guide future improvements and evaluation.

Current Progress and Next Steps

The project successfully establishes a working pipeline from data processing to forecasting and decision-making.

Completed:

- Data preprocessing and EDA
- Baseline machine learning model (Linear Regression)
- Initial deep learning model (CNN/TCN)
- NLP prototype module
- RL agent stub with reward design

Next Steps:

- Improve deep learning model performance (LSTM/TCN tuning)
- Integrate RL with real or forecasted demand data
- Perform ablation studies and error/slice analysis
- Expand NLP component functionality

- Finalize ethics and model documentation